

Zadania

5.1 Pod adresem <http://127.0.0.1:5001> działa prosty serwer HTTP udostępniający 2 endpointy: <http://127.0.0.1:5001/submit/public> oraz <http://127.0.0.1:5001/submit/private>.

(a) Uruchom serwer za pomocą poniższego polecenia:

```
docker run -p 5001:5001 --name ex1 docker.io/mazurkatarzyna/gpg-ex1:latest
podman run -p 5001:5001 --name ex1 docker.io/mazurkatarzyna/gpg-ex1:latest
```

Jeśli Docker Hub nie odpowiada, użyj obrazu zapasowego:

```
docker run -p 5001:5001 --name ex1 ghcr.io/mazurkatarzynaumcs/gpg-ex1:latest
podman run -p 5001:5001 --name ex1 ghcr.io/mazurkatarzynaumcs/gpg-ex1:latest
```

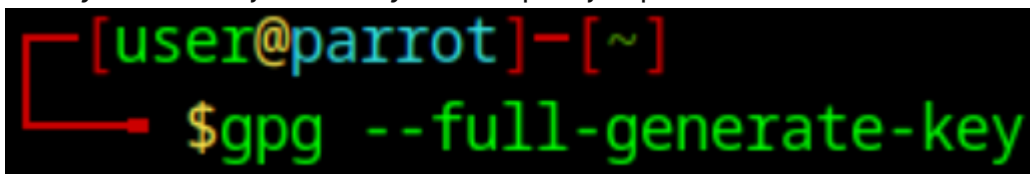
- (b) Wygeneruj parę kluczy GPG używając algorytmu RSA.
- (c) Wyeksportuj klucz publiczny i klucz prywatny do plików, odpowiednio `pub.key` oraz `priv.key`.
- (d) Wyślij request do endpointa <http://127.0.0.1:5001/submit/public> używając metody HTTP POST i prześlij na serwer wyeksportowany klucz publiczny GPG. Serwer w odpowiedzi zwróci informacje o kluczu.
- (e) Wyślij request do endpointa <http://127.0.0.1:5001/submit/private> używając metody HTTP POST i prześlij na serwer wyeksportowany klucz prywatny GPG. Serwer w odpowiedzi zwróci informacje o kluczu.

UWAGI:

- Aby za pomocą narzędzia `cURL` wysłać plik do serwera, użyj składni: `-F "plik=@hello.txt"`

a)

0. Po wywołaniu tej komendy trzeba przejść przez kilka kroków tworzenia klucza



```
[user@parrot]~$ gpg --full-generate-key
```

1. Wybieramy jaki rodzaj klucza chcemy. W tym zadaniu RSA więc opcja 1

```
Please select what kind of key you want:
(1) RSA and RSA (default)
(2) DSA and Elgamal
(3) DSA (sign only)
(4) RSA (sign only)
(14) Existing key from card
Your selection? 1
```

2. Wybieramy długość klucza. W poleceniu nie ma żadnych wytycznych więc można przykładowo 1024

```
RSA keys may be between 1024 and 4096 bits long.
What keysize do you want? (3072) 1024
Requested keysize is 1024 bits
```

3. Wybieramy jak długo klucz będzie aktywny. Ja daję jeden dzień bo nie ma żadnych informacji w poleceniu o tym

```
Please specify how long the key should be valid.
    0 = key does not expire
    <n> = key expires in n days
    <n>w = key expires in n weeks
    <n>m = key expires in n months
    <n>y = key expires in n years
Key is valid for? (0) 1
Key expires at Sun Nov 23 09:14:36 2025 UTC
Is this correct? (y/N) y
```

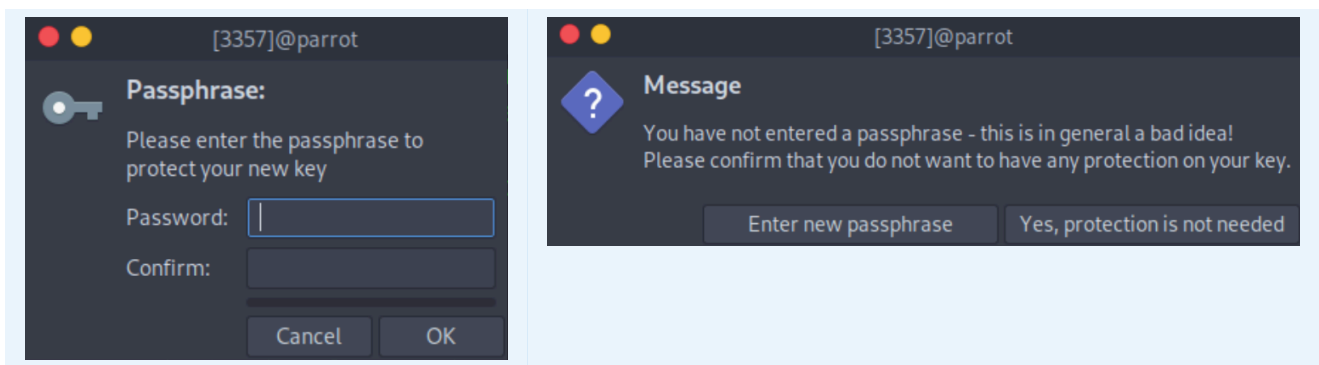
4. Wpisujemy dane do klucza

```
GnuPG needs to construct a user ID to identify your key.

Real name: student1
Email address: student1@mail.com
Comment:
You selected this USER-ID:
    "student1 <student1@mail.com>"

Change (N)ame, (C)omment, (E)mail or (O)kay/(Q)uit? 0
```

5. Wpisujemy hasło do zabezpieczenia klucza (można zostawić puste hasło i kliknąć zatwierdzenie, czasem prosi o potwierdzenie 2 razy)



6. Na koniec dostajemy informacje związane z kluczem

```
pub  rsa1024 2025-11-22 [SC] [expires: 2025-11-23]
     686255A5989FACDF4C2F2771A28306E705B5CB6A
uid                               student1 <student1@mail.com>
sub  rsa1024 2025-11-22 [E] [expires: 2025-11-23]
```

b)

Klucz możemy wyeksportować przez jego id lub email podany przy tworzeniu (przez email może się krzaczyć jak jest kilka bo wybiera sobie samo ten najbardziej aktualny i nie ma pewności że weźmie ten co chcemy)

Trzeba podać --armor bo w kluczu będą krzaki których serwer nie zaakceptuje

```
[user@parrot]~$ gpg --armor --export 686255A5989FACDF4C2F2771A28306E705B5CB6A > pub.key
```

```
[user@parrot]~$ gpg --armor --export student1@mail.com > pub.key
```

```
[user@parrot]~  
$gpg --armor --export-secret-key 686255A5989FACDF4C2F2771A28306E705B5CB6A > priv.key
```

c)

```
[user@parrot]~  
$curl -X POST http://localhost:5001/submit/public -F "file=@pub.key"  
{  
  "algo": "1",  
  "created": "1763802914",  
  "expires": "1763889314",  
  "fingerprint": "686255A5989FACDF4C2F2771A28306E705B5CB6A",  
  "key_id": "A28306E705B5CB6A",  
  "length": "1024",  
  "trust": "-",  
  "type": "public",  
  "uids": [  
    "student1 <student1@mail.com>"  
  ]  
}
```

d)

```
[user@parrot]~  
$curl -X POST http://localhost:5001/submit/private -F "file=@priv.key"  
{  
  "algo": "1",  
  "created": "1763802914",  
  "expires": "1763889314",  
  "fingerprint": "686255A5989FACDF4C2F2771A28306E705B5CB6A",  
  "key_id": "A28306E705B5CB6A",  
  "length": "1024",  
  "trust": "-",  
  "type": "private",  
  "uids": [  
    "student1 <student1@mail.com>"  
  ]  
}
```

5.2 Pod adresem <http://127.0.0.1:5002> działa prosty serwer HTTP udostępniający 2 endpointy: <http://127.0.0.1:5002/submit/public> oraz <http://127.0.0.1:5002/submit/private>.

(a) Uruchom serwer za pomocą poniższego polecenia:

```
docker run -p 5002:5002 --name ex2 docker.io/mazurkatarzyna/gpg-ex2:latest
podman run -p 5002:5002 --name ex2 docker.io/mazurkatarzyna/gpg-ex2:latest
```

Jeśli Docker Hub nie odpowiada, użyj obrazu zapasowego:

```
docker run -p 5002:5002 --name ex2 ghcr.io/mazurkatarzynaumcs/gpg-ex2:latest
podman run -p 5002:5002 --name ex2 ghcr.io/mazurkatarzynaumcs/gpg-ex2:latest
```

- (b) Wygeneruj parę kluczy GPG używając algorytmu DSA.
- (c) Wyeksportuj klucz publiczny i klucz prywatny do plików, odpowiednio `pub.key` oraz `priv.key`.
- (d) Wyślij request do endpointa <http://127.0.0.1:5002/submit/public> używając metody HTTP POST i prześlij na serwer wyeksportowany klucz publiczny GPG. Serwer w odpowiedzi zwróci informacje o kluczu.
- (e) Wyślij request do endpointa <http://127.0.0.1:5002/submit/private> używając metody HTTP POST i prześlij na serwer wyeksportowany klucz prywatny GPG. Serwer w odpowiedzi zwróci informacje o kluczu.

UWAGI:

- Aby za pomocą narzędzia cURL wysłać plik do serwera, użyj składni: `-F "plik=@hello.txt"`

Wybieramy typ klucza 2 bo w poleceniu jest informacja o tym, że ma być DSA

```
[user@parrot]~$ gpg --full-generate-key
gpg (GnuPG) 2.2.40; Copyright (C) 2022 g10 Code GmbH
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

Please select what kind of key you want:
  (1) RSA and RSA (default)
  (2) DSA and Elgamal
  (3) DSA (sign only)
  (4) RSA (sign only)
 (14) Existing key from card
Your selection? 2
```

```
pub   dsa1024 2025-11-22 [SC] [expires: 2025-11-23]
      FF7599C37737B5EA743612F549F6084E215A1BE8
uid           student2 <student2@mail.com>
sub    elg1024 2025-11-22 [E] [expires: 2025-11-23]

[user@parrot]~$ gpg --armor --export FF7599C37737B5EA743612F549F6084E215A1BE8 > pub.key
[user@parrot]~$ gpg --armor --export-secret-key FF7599C37737B5EA743612F549F6084E215A1BE8 > priv.key
```

```
[user@parrot]-[~]
$curl -X POST http://localhost:5002/submit/public -F "file=@pub.key"
{
  "algorithm": "DSA",
  "created": "1763803955",
  "expires": "1763890355",
  "fingerprint": "FF7599C37737B5EA743612F549F6084E215A1BE8",
  "format": "GPG/OpenPGP",
  "key_id": "49F6084E215A1BE8",
  "key_size": 1024,
  "trust": "-",
  "type": "public",
  "uids": [
    "student2 <student2@mail.com>"
  ]
}
[user@parrot]-[~]
$curl -X POST http://localhost:5002/submit/private -F "file=@priv.key"
{
  "algorithm": "DSA",
  "created": "1763803955",
  "expires": "1763890355",
  "fingerprint": "FF7599C37737B5EA743612F549F6084E215A1BE8",
  "format": "GPG/OpenPGP",
  "key_id": "49F6084E215A1BE8",
  "key_size": 1024,
  "trust": "-",
  "type": "private",
  "uids": [
    "student2 <student2@mail.com>"
  ]
}
```

5.3 Pod adresem <http://127.0.0.1:5003> działa prosty serwer HTTP udostępniający 2 endpointy: <http://127.0.0.1:5003/submit/public> oraz <http://127.0.0.1:5003/submit/private>.

(a) Uruchom serwer za pomocą poniższego polecenia:

```
docker run -p 5003:5003 --name ex3 docker.io/mazurkatarzyna/gpg-ex3:latest
podman run -p 5003:5003 --name ex3 docker.io/mazurkatarzyna/gpg-ex3:latest
```

Jeśli Docker Hub nie odpowiada, użyj obrazu zapasowego:

```
docker run -p 5003:5003 --name ex3 ghcr.io/mazurkatarzynaumcs/gpg-ex3:latest
podman run -p 5003:5003 --name ex3 ghcr.io/mazurkatarzynaumcs/gpg-ex3:latest
```

- (b) Wygeneruj parę kluczy GPG używając algorytmu RSA. Klucze powinny mieć 1024 bity długości, mają być ważne rok, oraz być wygenerowane dla adresu `student@uczelnia.pl`.
- (c) Wyeksportuj klucz publiczny i klucz prywatny do plików, odpowiednio `pub.key` oraz `priv.key`.
- (d) Wyślij request do endpointa <http://127.0.0.1:5003/submit/public> używając metody HTTP POST i prześlij na serwer wyeksportowany klucz publiczny GPG. Serwer w odpowiedzi zwróci informacje o kluczu.
- (e) Wyślij request do endpointa <http://127.0.0.1:5003/submit/private> używając metody HTTP POST i prześlij na serwer wyeksportowany klucz prywatny GPG. Serwer w odpowiedzi zwróci informacje o kluczu.

```
[user@parrot]~$ gpg --full-generate-key
```

```
Please select what kind of key you want:
(1) RSA and RSA (default)
(2) DSA and Elgamal
(3) DSA (sign only)
(4) RSA (sign only)
(14) Existing key from card
Your selection? 1
```

```
RSA keys may be between 1024 and 4096 bits long.
What keysize do you want? (3072) 1024
```

Please specify how long the key should be valid.

0 = key does not expire

<n> = key expires in n days

<n>w = key expires in n weeks

<n>m = key expires in n months

<n>y = key expires in n years

Key is valid for? (0) 1y

Key expires at Sun Nov 22 09:38:41 2026 UTC

```
pub  rsa1024 2025-11-22 [SC] [expires: 2026-11-22]
     E55935F672FA6199DB05AC47E68A27AFF4E7390B
```

```
uid          student3 <student@uczelnia.pl>
```

```
sub  rsa1024 2025-11-22 [E] [expires: 2026-11-22]
```

```
[user@parrot]~
```

```
$gpg --armor --export E55935F672FA6199DB05AC47E68A27AFF4E7390B > pub.key
```

```
[user@parrot]~
```

```
$gpg --armor --export-secret-key E55935F672FA6199DB05AC47E68A27AFF4E7390B > priv.key
```

```
[user@parrot]-[~]
└─$ curl -X POST http://localhost:5003/submit/public -F "file=@pub.key"
{
  "algo": "1",
  "created": "1763804343",
  "expires": "1795340343",
  "fingerprint": "E55935F672FA6199DB05AC47E68A27AFF4E7390B",
  "key_id": "E68A27AFF4E7390B",
  "length": "1024",
  "trust": "-",
  "type": "public",
  "uids": [
    "student3 <student@uczelnia.pl>"
  ],
  "validation": "passed"
}
[user@parrot]-[~]
└─$ curl -X POST http://localhost:5003/submit/private -F "file=@priv.key"
{
  "algo": "1",
  "created": "1763804343",
  "expires": "1795340343",
  "fingerprint": "E55935F672FA6199DB05AC47E68A27AFF4E7390B",
  "key_id": "E68A27AFF4E7390B",
  "length": "1024",
  "trust": "-",
  "type": "private",
  "uids": [
    "student3 <student@uczelnia.pl>"
  ],
  "validation": "passed"
}
```

5.4 Pod adresem <http://127.0.0.1:5004> działa prosty serwer HTTP udostępniający 2 endpointy: <http://127.0.0.1:5004/decrypt> oraz <http://127.0.0.1:5004/submit>.

(a) Uruchom serwer za pomocą poniższego polecenia:

```
docker run -p 5004:5004 --name ex4 docker.io/mazurkatarzyna/gpg-ex4:latest
podman run -p 5004:5004 --name ex4 docker.io/mazurkatarzyna/gpg-ex4:latest
```

Jeśli Docker Hub nie odpowiada, użyj obrazu zapasowego:

```
docker run -p 5004:5004 --name ex4 ghcr.io/mazurkatarzynaumcs/gpg-ex4:latest
podman run -p 5004:5004 --name ex4 ghcr.io/mazurkatarzynaumcs/gpg-ex4:latest
```

- (b) Wyślij request do endpointa <http://127.0.0.1:5004/decrypt> używając metody HTTP GET i zapisz odpowiedź do pliku z rozszerzeniem *.zip.
- (c) Rozpakuj pobrane archiwum.
- (d) Odszyfruj plik `encrypted.txt` używając algorytmu CAMELLIA128 i hasła znajdującego się w pliku `passphrase.txt`.
- (e) Wyślij request do endpointa <http://127.0.0.1:5004/submit/> używając metody HTTP POST, i prześlij do serwera ID sesji (jako `session_id`, które znajdziesz w pliku `session_id.txt`) oraz odszyfrowane słowo (jako `decrypted_text`). Serwer w odpowiedzi zwróci informacje o poprawnym lub niepoprawnym deszyfrowaniu.

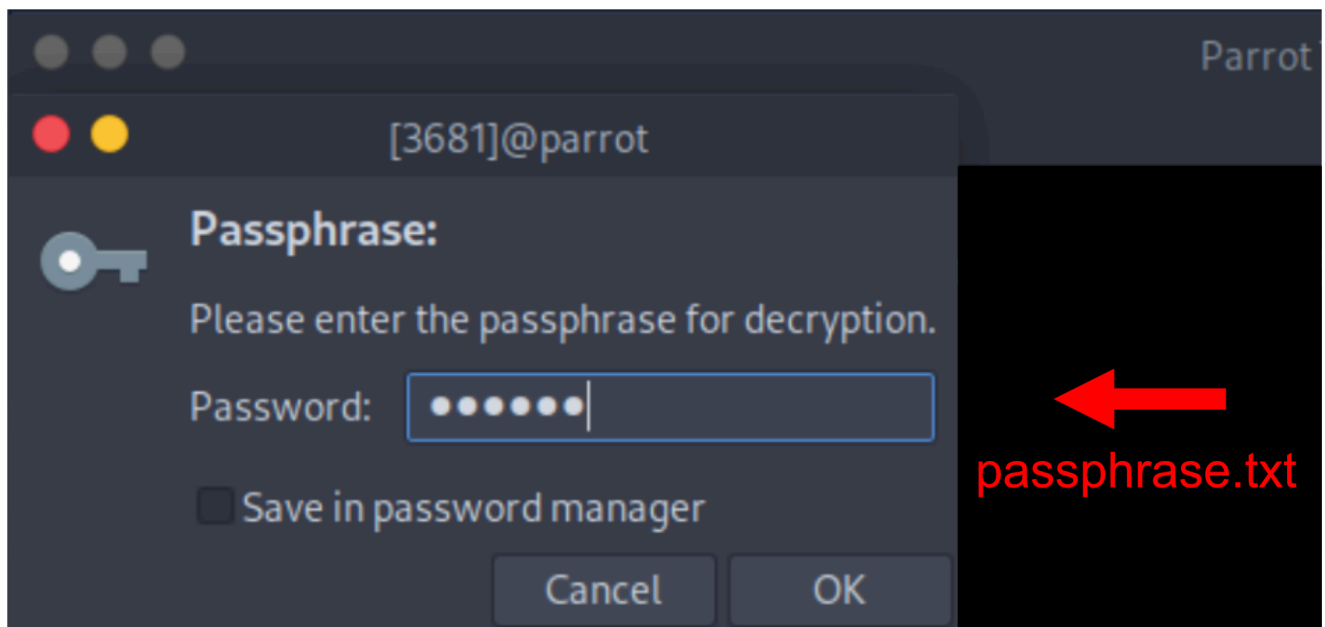
UWAGI:

- Aby zapisać odpowiedź do pliku z rozszerzeniem *.zip, użyj opcji -o.
- Podczas deszyfrowania symetrycznego sprawdź opcje:
 - cipher-algo
 - batch --yes
 - passphrase

```
[user@parrot]~$ curl -s -o 5_4.zip -X GET http://localhost:5004/decrypt
[user@parrot]~$ unzip 5_4
Archive: 5_4.zip
replace encrypted.txt? [y]es, [n]o, [A]ll, [N]one, [r]ename: A
  inflating: encrypted.txt
  inflating: passphrase.txt
  inflating: session_id.txt
```

```
[x]-[user@parrot]~$ cat passphrase.txt
123456
[user@parrot]~$ cat session_id.txt
6fd8cd87310ab1a0a6269a16f31d6688
[user@parrot]~$
```

```
$gpg --cipher-algo CAMELLIA128 --decrypt encrypted.txt > decrypted.txt
gpg: CAMELLIA128.CFB encrypted data
gpg: encrypted with 1 passphrase
```



```
[user@parrot]~$cat decrypted.txt
welcome_
```

```
[user@parrot]~$curl -X POST http://localhost:5004/submit -H "Content-Type: application/json" -d '{"session_id":"6fd8cd87310ab1a0a6269a16f31d6688","decrypted_text":"welcome"}'
{"message": "Gratulacje! Poprawnie odszyfrowa\u0142e\u015b plik!",
"passphrase_used": "123456",
"secret_word": "welcome",
"status": "success"}
```

5.5 Pod adresem <http://127.0.0.1:5005> działa prosty serwer HTTP udostępniający 2 endpointy: <http://127.0.0.1:5005/encrypt> oraz <http://127.0.0.1:5005/submit>.

(a) Uruchom serwer za pomocą poniższego polecenia:

```
docker run -p 5005:5005 --name ex5 docker.io/mazurkatarzyna/gpg-ex5:latest
podman run -p 5005:5005 --name ex5 docker.io/mazurkatarzyna/gpg-ex5:latest
```

Jeśli Docker Hub nie odpowiada, użyj obrazu zapasowego:

```
docker run -p 5005:5005 --name ex5 ghcr.io/mazurkatarzynaumcs/gpg-ex5:latest
podman run -p 5005:5005 --name ex5 ghcr.io/mazurkatarzynaumcs/gpg-ex5:latest
```

- (b) Wyślij request do endpointa <http://127.0.0.1:5005/encrypt> używając metody HTTP GET. W odpowiedzi od serwera otrzymasz słowo do zaszyfrowania (jako `word_to_encrypt`), hasło potrzebne do szyfrowania (jako `passphrase`) oraz id sesji (jako `session_id`).
- (c) Zaszyfruj odebrane od serwera słowo (`word_to_encrypt`) za pomocą algorytmu AES128 oraz odebranego od serwera hasła (`passphrase`). Zaszyfrowane słowo ma być również zakodowane przy pomocy kodowania base64.
- (d) Wyślij request do endpointa <http://127.0.0.1:5005/submit> używając metody HTTP POST i prześlij do serwera id sesji (jako `session_id`) oraz zaszyfrowany i zakodowany plik (jako plik, `encrypted_file`).

UWAGI:

- Aby wykorzystać kodowanie base64 podczas szyfrowania, użyj opcji `--armor`,
- Aby za pomocą narzędzia cURL wysłać plik do serwera, użyj składni: `-F "plik=@hello.txt"`

```
[user@parrot]~$ curl -X GET http://localhost:5005/encrypt
{
  "session_id": "6e6806a9915a08e93afffd57bfb5e34a",
  "word_to_encrypt": "marcy",
  "passphrase": "moose",
  "algorithm": "AES128",
  "instructions": "Użyj GPG do zaszyfrowania słowa algorytmem AES128 z podanym hasłem. Następnie wyślij zaszyfrowany plik do /submit wraz z session_id."
}
[user@parrot]~$ echo -n "marcy" > word_to_encrypt.txt
[user@parrot]~$ gpg --symmetric --cipher-algo AES128 --armor --output encrypted.txt word_to_encrypt.txt
```

```
[user@parrot]~$ curl -X POST http://localhost:5005/submit -F "session_id=6e6806a9915a08e93afffd57bfb5e34a" -F "encrypted_file=@encrypted.txt"
{
  "status": "success",
  "message": "Gratulacje! Poprawnie zaszyfrowałeś plik!",
  "word": "marcy",
  "passphrase_used": "moose"
}
[user@parrot]~$
```


5.6 Pod adresem <http://127.0.0.1:5006> działa prosty serwer HTTP udostępniający 2 endpointy: <http://127.0.0.1:5006/encrypt> oraz <http://127.0.0.1:5006/submit>.

(a) Uruchom serwer za pomocą poniższego polecenia:

```
docker run -p 5006:5006 --name ex6 docker.io/mazurkatarzyna/gpg-ex6:latest
podman run -p 5006:5006 --name ex6 docker.io/mazurkatarzyna/gpg-ex6:latest
```

Jeśli Docker Hub nie odpowiada, użyj obrazu zapasowego:

```
docker run -p 5006:5006 --name ex6 ghcr.io/mazurkatarzynaumcs/gpg-ex6:latest
podman run -p 5006:5006 --name ex6 ghcr.io/mazurkatarzynaumcs/gpg-ex6:latest
```

- (b) Wyślij request do endpointa <http://127.0.0.1:5005/encrypt> używając metody HTTP GET i zapisz odpowiedź do pliku z rozszerzeniem *.zip.
- (c) Rozpakuj pobrane archiwum. W odpowiedzi od serwera otrzymałeś id sesji (jako `session_id`), klucz publiczny GPG (`public_key.asc`), klucz prywatny GPG (`private_key.asc`), oraz słowo do zaszyfrowania (`word.txt`).
- (d) Zaimportuj do swojego zbioru kluczy klucz publiczny pobrany od serwera.
- (e) Wykorzystując narzędzie `gpg` oraz pobrany klucz publiczny (z pliku `public_key.asc`), zaszyfruj odebrane od serwera słowo (podczas szyfrowania nie używaj hasła).
- (f) Zaszyfrowane słowo zakoduj w base64 i zapisz do pliku.
- (g) Wyślij request do endpointa <http://127.0.0.1:5006/submit> używając metody HTTP POST i prześlij do serwera id sesji (jako `session_id`) oraz zaszyfrowane i zakodowane słowo (jako plik, `encrypted_file`).

Bezpieczeństwo Systemów Komputerowych - GNU Privacy Guard (GPG) | Katarzyna Mazur

UWAGI:

- Aby zapisać odpowiedź do pliku z rozszerzeniem *.zip, użyj opcji `-o`.
- Aby wykorzystać kodowanie base64 podczas szyfrowania, użyj opcji `--armor`,
- Aby za pomocą narzędzia `cURL` wysłać plik do serwera, użyj składni: `-F "plik=@hello.txt"`

```
[user@parrot]~  
└─$ curl -s -o 5_6.zip -X GET http://localhost:5006/encrypt  
[user@parrot]~  
└─$ unzip 5_6.zip  
Archive: 5_6.zip  
replace word.txt? [y]es, [n]o, [A]ll, [N]one, [r]ename: A  
  inflating: word.txt  
  inflating: public_key.asc  
  inflating: private_key.asc  
  inflating: session_id.txt  
[user@parrot]~  
└─$ cat session_id.txt  
b2cc719f660e33b095ce671a7a76ed17 [user@parrot]~  
└─$ gpg --import public_key.asc  
gpg: key 0322A1FAB4083754: public key "Challenge User <challengex6@example.com>" imported  
gpg: Total number processed: 1  
gpg:          imported: 1
```

```
[user@parrot]~  
└─$ gpg --list-keys
```

```
pub      rsa2048 2025-11-22 [SCEA]  
          BC61750B6439EF6CDE0388560322A1FAB4083754  
uid              [ unknown] Challenge User <challengex6@example.com>
```

```
[user@parrot]~  
└─$ gpg --encrypt --recipient BC61750B6439EF6CDE0388560322A1FAB4083754 --armor --output encrypted.txt word.txt
```

```
[user@parrot]~  
└─$ cat encrypted.txt  
-----BEGIN PGP MESSAGE-----  
  
hQEMAwMiofq0CDdUAQf7B/avRXs1AeeUtjdMyf60B3oC+I10PRGMfIn6XoXcS0ot  
hlGsgMH5tv4C7Wi870D5f0ku2lSzTQkbQcI+9M4UKkKyoiou3hRGp7xi3Bgag5ha  
W6+UjW+kgKVBKvCy/vvP96EzBW8PYBJR7u8EtBX0apRjP4gWHQ9HkQ6UQZi37jH  
6QFhY0c5SCp0kopPA0d05mkWgDM4Hc9Gw9g3AbzbHpjzE+PmJr59MPc54NA8B5zB  
lvxT0BCKfSXo7bjN9oyQisy0xojl5iVPtXZF2Wtrncqxi9qaLfobtTqf1qLnvtwG  
2Krc1L3sTxXTBgUrn4tb8rXsD2pPqedMYHgg5qIgd9JIATE8Nu+eZv22aSmZlATU  
4jjttSlsvHPg2NHaDRqfcOwLh/7zGwlma1CpVxWRWvgcE1H0SLEJ8qeMxYJc4XO  
/puoe5XWUNZA  
=cq+Q  
-----END PGP MESSAGE-----
```

```
[user@parrot]~$ curl -X POST http://localhost:5006/submit -F "session_id=b2cc719f660e33b095ce671a7a76ed17" -F "encrypted_file=@encrypted.txt"
{"status": "success",
 "message": "Gratulacje! Poprawnie zaszyfrowałeś plik!",
 "word": "beryl"}
[user@parrot]~$
```

5.7 Pod adresem <http://127.0.0.1:5007> działa prosty serwer HTTP udostępniający 2 endpointy: <http://127.0.0.1:5007/decrypt> oraz <http://127.0.0.1:5007/submit>.

- (a) Uruchom serwer za pomocą poniższego polecenia:

```
docker run -p 5007:5007 --name ex7 docker.io/mazurkatarzyna/gpg-ex7:latest
podman run -p 5007:5007 --name ex7 docker.io/mazurkatarzyna/gpg-ex7:latest
```

Jeśli Docker Hub nie odpowiada, użyj obrazu zapasowego:

```
docker run -p 5007:5007 --name ex7 ghcr.io/mazurkatarzynaumcs/gpg-ex7:latest
podman run -p 5007:5007 --name ex7 ghcr.io/mazurkatarzynaumcs/gpg-ex7:latest
```

- (b) Wyślij request do endpointa <http://127.0.0.1:5007/decrypt> używając metody HTTP GET i zapisz odpowiedź do pliku z rozszerzeniem *.zip.
- (c) Rozpakuj pobrane archiwum. W odpowiedzi od serwera otrzymałeś id sesji (jako `session_id`), klucz publiczny GPG (`public_key.asc`), klucz prywatny GPG (`private_key.asc`), oraz słowo do odszyfrowania (`encrypted_word.asc`).
- (d) Odszyfruj odebrane od serwera słowo (`encrypted_word.asc`) za pomocą pobranego klucza prywatnego (`private_key.asc`).
- (e) Wyślij request do endpointa <http://127.0.0.1:5007/submit> używając metody HTTP POST i prześlij do serwera id sesji (jako `session_id`) oraz odszyfrowane słowo (jako tekst, `decrypted_word`).

UWAGI:

- Aby zapisać odpowiedź do pliku z rozszerzeniem *.zip, użyj opcji -o

```
[user@parrot]-[~]
└─ $curl -s -o 5_7.zip -X GET http://localhost:5007/decrypt
[user@parrot]-[~]
└─ $unzip 5_7.zip
Archive: 5_7.zip
replace public_key.asc? [y]es, [n]o, [A]ll, [N]one, [r]ename: A
  inflating: public_key.asc
  inflating: private_key.asc
  inflating: encrypted_word.asc
  inflating: session_id.txt
  inflating: README.txt
[user@parrot]-[~]
└─ $cat session_id.txt
7b5393f86a3b0b65204ba1f9b8902e5d [user@parrot]-[~]
└─ $cat README.txt
GPG Decryption Challenge
=====

1. Użyj klucza prywatnego (private_key.asc) do odszyfrowania pliku encrypted_word.asc
2. Wyślij odszyfrowane słowo na endpoint /submit wraz z session_id

Session ID: 7b5393f86a3b0b65204ba1f9b8902e5d
[user@parrot]-[~]
└─ $gpg --import private_key.asc
gpg: key FD0A47298CE9BE2B: public key "Decrypt Challenge <decrypt@example.com>" imported
gpg: key FD0A47298CE9BE2B: secret key imported
gpg: Total number processed: 1
gpg:      imported: 1
gpg:      secret keys read: 1
gpg:      secret keys imported: 1
```

```
[user@parrot]-[~]
└─ $gpg --list-keys
pub  rsa2048 2025-11-22 [SCEA]
    981F974D6BAC4550231E5A08FD0A47298CE9BE2B
uid  [ unknown] Decrypt Challenge <decrypt@example.com>
```

```
[x]-[user@parrot]-[~]
└─ $gpg --decrypt --recipient 981F974D6BAC4550231E5A08FD0A47298CE9BE2B --output decrypted_word.txt encrypted_word.asc
gpg: encrypted with 2048-bit RSA key, ID FD0A47298CE9BE2B, created 2025-11-22
    "Decrypt Challenge <decrypt@example.com>"
[user@parrot]-[~]
└─ $cat decrypted_word.txt
simpsons [user@parrot]-[~]
```

```
[user@parrot]-[~]
└─ $curl -X POST http://localhost:5007/submit -F "session_id=7b5393f86a3b0b65204ba1f9b8902e5d" -F "decrypted_word=simpsons"
{
  "status": "success",
  "message": "🎉 Gratulacje! Poprawnie odszyfrowałeś słowo!",
  "word": "simpsons"
} [user@parrot]-[~]
```

5.8 Pod adresem <http://127.0.0.1:5008> działa prosty serwer HTTP udostępniający 2 endpointy: <http://127.0.0.1:5008/sign> oraz <http://127.0.0.1:5008/submit>.

(a) Uruchom serwer za pomocą poniższego polecenia:

```
docker run -p 5008:5008 --name ex8 docker.io/mazurkatarzyna/gpg-ex8:latest
podman run -p 5008:5008 --name ex8 docker.io/mazurkatarzyna/gpg-ex8:latest
```

Jeśli Docker Hub nie odpowiada, użyj obrazu zapasowego:

```
docker run -p 5008:5008 --name ex8 ghcr.io/mazurkatarzynaumcs/gpg-ex8:latest
podman run -p 5008:5008 --name ex8 ghcr.io/mazurkatarzynaumcs/gpg-ex8:latest
```

- (b) Wyślij request do endpointa <http://127.0.0.1:5008/sign> używając metody HTTP GET i zapisz odpowiedź do pliku z rozszerzeniem *.zip.
- (c) Rozpakuj pobrane archiwum. W odpowiedzi od serwera otrzymałeś id sesji (jako `session_id`), klucz publiczny GPG (`public_key.asc`), klucz prywatny GPG (`private_key.asc`), oraz słowo do podpisania (w pliku `word.txt`).
- (d) Podpisz plik (`word.txt`) za pomocą pobranego klucza publicznego (`public_key.asc`), tworząc plik `word.txt.gpg`.

Bezpieczeństwo Systemów Komputerowych - GNU Privacy Guard (GPG) | Katarzyna Mazur

- (e) Wyślij request do endpointa <http://127.0.0.1:5008/submit> używając metody HTTP POST i prześlij do serwera id sesji (jako `session_id`) oraz podpisane słowo (jako plik, `signed_file`).

UWAGI:

- Aby zapisać odpowiedź do pliku z rozszerzeniem *.zip, użyj opcji -o,
- Aby utworzyć podpis zintegrowany, użyj opcji --sign,
- Aby wskazać klucz, którego będziemy używać do podpisywania, sprawdź opcję -u,
- Aby za pomocą narzędzia cURL wysłać plik do serwera, użyj składni: -F "plik=@hello.txt"

```
[user@parrot]~  
└─$ curl -s -o 5_8.zip -X GET http://localhost:5008/sign  
[user@parrot]~  
└─$ unzip 5_8.zip  
Archive: 5_8.zip  
replace word.txt? [y]es, [n]o, [A]ll, [N]one, [r]ename: A  
  inflating: word.txt  
  inflating: public_key.asc  
  inflating: private_key.asc  
  inflating: session_id.txt  
[user@parrot]~  
└─$ cat session_id.txt  
36e462cebd5978e58418471a64d1613b [user@parrot]~  
└─$ gpg --import private_key.asc  
gpg: key F4793C047AA44820: public key "Sign Challenge <sign@example.com>" imported  
gpg: key F4793C047AA44820: secret key imported  
gpg: Total number processed: 1  
gpg:      imported: 1  
gpg:      secret keys read: 1  
gpg:      secret keys imported: 1  
[user@parrot]~  
└─$ gpg --list-keys
```

```
pub  rsa2048 2025-11-22 [SCEA]  
    57961D73D18AF402B4023AC8F4793C047AA44820  
uid          [ unknown] Sign Challenge <sign@example.com>  
  
[user@parrot]~  
└─$ gpg -u 57961D73D18AF402B4023AC8F4793C047AA44820 --sign --output word.txt.gpg word.txt  
File 'word.txt.gpg' exists. Overwrite? (y/N) y  
[user@parrot]~  
└─$ curl -X POST http://localhost:5008/submit -F "session_id=36e462cebd5978e58418471a64d1613b" -F "signed_file=@word.txt.gpg"  
{  
  "status": "success",  
  "message": "🎉 Gratulacje! Poprawnie podpisałeś plik!",  
  "word": "aria",  
  "signer": "Sign Challenge <sign@example.com>",  
  "signature_valid": true  
}[user@parrot]~
```

5.9 Pod adresem <http://127.0.0.1:5009> działa prosty serwer HTTP udostępniający 2 endpointy: <http://127.0.0.1:5009/detach-sign> oraz <http://127.0.0.1:5009/submit>.

(a) Uruchom serwer za pomocą poniższego polecenia:

```
docker run -p 5009:5009 --name ex9 docker.io/mazurkatarzyna/gpg-ex9:latest
podman run -p 5009:5009 --name ex9 docker.io/mazurkatarzyna/gpg-ex9:latest
```

Jeśli Docker Hub nie odpowiada, użyj obrazu zapasowego:

```
docker run -p 5009:5009 --name ex9 ghcr.io/mazurkatarzynaumcs/gpg-ex9:latest
podman run -p 5009:5009 --name ex9 ghcr.io/mazurkatarzynaumcs/gpg-ex9:latest
```

- (b) Wyślij request do endpointa <http://127.0.0.1:5009/detach-sign> używając metody HTTP GET i zapisz odpowiedź do pliku z rozszerzeniem *.zip.
- (c) Rozpakuj pobrane archiwum. W odpowiedzi od serwera otrzymałeś id sesji (jako `session_id`), klucz publiczny GPG (`public_key.asc`), klucz prywatny GPG (`private_key.asc`), oraz słowo do podpisania (w pliku `word.txt`).
- (d) Podpisz plik (`word.txt`) za pomocą pobranego klucza publicznego (`public_key.asc`), tworząc plik `word.txt.sig`.
- (e) Wyślij request do endpointa <http://127.0.0.1:5009/submit> używając metody HTTP POST i prześlij do serwera id sesji (jako `session_id`), plik ze słowem (jako plik, `original_file`) oraz podpisane słowo (jako plik, `signature_file`).

UWAGI:

- Aby zapisać odpowiedź do pliku z rozszerzeniem *.zip, użyj opcji -o,
- Aby utworzyć oddzielny podpis, użyj opcji --detach-sign,
- Aby wskazać klucz, którego będziemy używać do podpisywania, sprawdź opcję -u,
- Aby za pomocą narzędzia cURL wysłać plik do serwera, użyj składni: -F "plik=@hello.txt"

```
!
[user@parrot]~$ curl -s -o 5_9.zip -X GET http://127.0.0.1:5009/detach-sign
[user@parrot]~$ unzip 5_9.zip
Archive: 5_9.zip
  inflating: word.txt
  inflating: public_key.asc
  inflating: private_key.asc
  inflating: session_id.txt
[user@parrot]~$ cat session_id.txt
5a9108c890ab46cb3d1744b0aee4f82a [user@parrot]~$
[user@parrot]~$ gpg --import private_key.asc
gpg: key 68670FDD1CA4695A: public key "Detach Sign Challenge <detach@example.com>" imported
gpg: key 68670FDD1CA4695A: secret key imported
gpg: Total number processed: 1
gpg:         imported: 1
gpg:     secret keys read: 1
gpg:     secret keys imported: 1
[user@parrot]~$ gpg --list-keys
```

```
pub  rsa2048 2025-11-22 [SCEA]
      5C4090E921CA3F366D8E7EEB68670FDD1CA4695A
uid      [ unknown] Detach Sign Challenge <detach@example.com>

[user@parrot]~]
└─$ gpg -u 5C4090E921CA3F366D8E7EEB68670FDD1CA4695A --detach-sign --output word.txt.sig word.txt
[user@parrot]~]
└─$ curl -X POST http://localhost:5009/submit -F "session_id=5a9108c890ab46cb3d1744b0aee4f82a" -F "original_file=@word.txt" -F "signature_file=@word.txt.sig"
{
  "status": "success",
  "message": "🎉 Gratulacje! Poprawnie utworzyłeś oddzielny podpis!",
  "word": "pam",
  "signer": "Detach Sign Challenge <detach@example.com>",
  "signature_valid": true,
  "signature_type": "detached"
}
[user@parrot]~]
```

5.10 Pod adresem <http://127.0.0.1:5010> działa prosty serwer HTTP udostępniający 2 endpointy: <http://127.0.0.1:5010/clearsign> oraz <http://127.0.0.1:5010/submit>.

(a) Uruchom serwer za pomocą poniższego polecenia:

```
docker run -p 5010:5010 --name ex10 docker.io/mazurkatarzyna/gpg-ex10:latest
podman run -p 5010:5010 --name ex10 docker.io/mazurkatarzyna/gpg-ex10:latest
```

Jeśli Docker Hub nie odpowiada, użyj obrazu zapasowego:

```
docker run -p 5010:5010 --name ex10 ghcr.io/mazurkatarzynaumcs/gpg-ex10:latest
podman run -p 5010:5010 --name ex10 ghcr.io/mazurkatarzynaumcs/gpg-ex10:latest
```

Bezpieczeństwo Systemów Komputerowych - GNU Privacy Guard (GPG) | Katarzyna Mazur

GNU Privacy Guard - GPG

Zadania

- (b) Wyślij request do endpointa <http://127.0.0.1:5010/clearsign> używając metody HTTP GET i zapisz odpowiedź do pliku z rozszerzeniem *.zip.
- (c) Rozpakuj pobrane archiwum. W odpowiedzi od serwera otrzymałeś id sesji (jako `session_id`), klucz publiczny GPG (`public_key.asc`), klucz prywatny GPG (`private_key.asc`), oraz słowo do podpisania (w pliku `word.txt`).
- (d) Podpisz plik (`word.txt`) za pomocą pobranego klucza publicznego (`public_key.asc`), tworząc plik `word.txt.asc`.
- (e) Wyślij request do endpointa <http://127.0.0.1:5010/submit> używając metody HTTP POST i prześlij do serwera id sesji (jako `session_id`) oraz podpisane słowo (jako plik, `clearsigned_file`).

UWAGI:

- Aby zapisać odpowiedź do pliku z rozszerzeniem *.zip, użyj opcji `-o`,
- Aby utworzyć czytelny podpis, użyj opcji `--clearsign`,
- Aby wskazać klucz, którego będziemy używać do podpisywania, sprawdź opcję `-u`,
- Aby za pomocą narzędzia cURL wysłać plik do serwera, użyj składni: `-F "plik=@hello.txt"`

!

```
[user@parrot]-[~]
└─$ curl -s -o 5_10.zip -X GET http://127.0.0.1:5010/clearsign
[user@parrot]-[~]
└─$ unzip 5_10.zip
Archive: 5_10.zip
  inflating: word.txt
  inflating: public_key.asc
  inflating: private_key.asc
  inflating: session_id.txt
[user@parrot]-[~]
└─$ cat session_id.txt
f86d26b152b41ac2f2d037de40539195 [user@parrot]-[~]
└─$ gpg --import private_key.asc
gpg: key 7A7A0C50087A1B2C: public key "Clearsign Challenge <clearsign@example.com>" imported
gpg: key 7A7A0C50087A1B2C: secret key imported
gpg: Total number processed: 1
gpg:         imported: 1
gpg:       secret keys read: 1
gpg:       secret keys imported: 1
[user@parrot]-[~]
└─$ gpg --list-keys
```

```
pub   rsa2048 2025-11-22 [SCEA]
      17FDD50EAF90E1F947E83C0A7A7A0C50087A1B2C
uid           [ unknown] Clearsign Challenge <clearsign@example.com>

[user@parrot]-[~]
└─$ gpg --local-user 17FDD50EAF90E1F947E83C0A7A7A0C50087A1B2C --clearsign word.txt
[user@parrot]-[~]
└─$ curl -X POST http://localhost:5010/submit -F "session_id=f86d26b152b41ac2f2d037de40539195" -F "clearsigned_file=@word.txt.asc"
{
  "status": "success",
  "message": "🎉 Gratulacje! Poprawnie utworzyłeś clear-signed plik!",
  "word": "subway",
  "signer": "Clearsign Challenge <clearsign@example.com>",
  "signature_valid": true,
  "signature_type": "clearsign",
  "note": "Treść była widoczna w pliku wraz z podpisem ASCII"
} [user@parrot]-[~]
```

5.11 Pod adresem <http://127.0.0.1:5011> działa prosty serwer HTTP udostępniający 2 endpointy: <http://127.0.0.1:5011/sign> oraz <http://127.0.0.1:5011/verify>.

(a) Uruchom serwer za pomocą poniższego polecenia:

```
docker run -p 5011:5011 --name ex11 docker.io/mazurkatarzyna/gpg-ex11:latest
podman run -p 5011:5011 --name ex11 docker.io/mazurkatarzyna/gpg-ex11:latest
```

Jeśli Docker Hub nie odpowiada, użyj obrazu zapasowego:

```
docker run -p 5011:5011 --name ex11 ghcr.io/mazurkatarzynaumcs/gpg-ex11:latest
podman run -p 5011:5011 --name ex11 ghcr.io/mazurkatarzynaumcs/gpg-ex11:latest
```

- (b) Wyślij request do endpointa <http://127.0.0.1:5011/sign> używając metody HTTP GET i zapisz odpowiedź do pliku z rozszerzeniem *.zip.
- (c) Rozpakuj pobrane archiwum. W odpowiedzi od serwera otrzymałeś id sesji (jako `session_id`), klucz publiczny GPG (`public_key.asc`), oraz podpisane słowo (w pliku `word.txt.gpg`).
- (d) Zweryfikuj podpis (plik `word.txt.gpg`) za pomocą pobranego klucza publicznego (`public_key.asc`). W przypadku, gdy weryfikacja się powiedzie, powinieneś zobaczyć adres e-mail osoby, która podpisała plik.
- (e) Wyślij request do endpointa <http://127.0.0.1:5011/verify> używając metody HTTP POST i prześlij do serwera id sesji (jako `session_id`) oraz adres e-mail podpisującego (jako tekst, `signer_email`).

UWAGI:

- Aby zapisać odpowiedź do pliku z rozszerzeniem *.zip, użyj opcji -o

```
[user@parrot]~$ curl -s -o data.zip -X GET http://127.0.0.1:5011/sign
[user@parrot]~$ unzip data.zip
Archive: data.zip
  inflating: word.txt.gpg
  inflating: public_key.asc
  inflating: session_id.txt
[user@parrot]~$ cat session_id.txt
bf81496e396e321874d5ed9ac33d5a51
[user@parrot]~$ gpg --import public_key.asc
gpg: key 1F45C54EC7171766: public key "Griffith <GRIFFITH@secure.com>" imported
gpg: Total number processed: 1
gpg:           imported: 1
[user@parrot]~$ gpg --list-keys
```

```
pub  rsa2048 2025-11-22 [SCEA]
      8991B43BF21C5215C7D683CD1F45C54EC7171766
uid   [ unknown] Griffith <GRIFFITH@secure.com>

[user@parrot]~$ gpg --verify word.txt.gpg
gpg: Signature made Sat Nov 22 13:41:35 2025 UTC
gpg:          using RSA key 8991B43BF21C5215C7D683CD1F45C54EC7171766
gpg: Good signature from "Griffith <GRIFFITH@secure.com>" [unknown]
gpg: WARNING: This key is not certified with a trusted signature!
gpg:          There is no indication that the signature belongs to the owner.
Primary key fingerprint: 8991 B43B F21C 5215 C7D6 83CD 1F45 C54E C717 1766

[user@parrot]~$ ^C

[x]-[user@parrot]~$ curl -X POST http://localhost:5011/verify -F "session_id=bf81496e396e321874d5ed9ac33d5a51" -F "signer_email=GRIFFITH@secure.com"
{
  "status": "success",
  "message": "🎉 Gratulacje! Poprawnie zweryfikowałeś podpis i zidentyfikowałeś podpisującego!",
  "signer_email": "GRIFFITH@secure.com",
  "word": "BARKER",
  "note": "Udowodniłeś że potrafisz zweryfikować podpis GPG i wydobyć informacje o podpisującym"
}
```

5.12 Pod adresem <http://127.0.0.1:5012> działa prosty serwer HTTP udostępniający 2 endpointy: <http://127.0.0.1:5012/detach-sign> oraz <http://127.0.0.1:5012/verify>.

(a) Uruchom serwer za pomocą poniższego polecenia:

```
docker run -p 5012:5012 --name ex12 docker.io/mazurkatarzyna/gpg-ex12:latest  
podman run -p 5012:5012 --name ex12 docker.io/mazurkatarzyna/gpg-ex12:latest
```

Bezpieczeństwo Systemów Komputerowych - GNU Privacy Guard (GPG) | Katarzyna Mazur

GNU Privacy Guard - GPG

Zadania

Jeśli Docker Hub nie odpowiada, użyj obrazu zapasowego:

```
docker run -p 5012:5012 --name ex12 ghcr.io/mazurkatarzynaumcs/gpg-ex12:latest  
podman run -p 5012:5012 --name ex12 ghcr.io/mazurkatarzynaumcs/gpg-ex12:latest
```

- (b) Wyślij request do endpointa <http://127.0.0.1:5012/sign> używając metody HTTP GET i zapisz odpowiedź do pliku z rozszerzeniem *.zip.
- (c) Rozpakuj pobrane archiwum. W odpowiedzi od serwera otrzymałeś id sesji (jako `session_id`), klucz publiczny GPG (`public_key.asc`), wylosowane słowo (w pliku `word.txt`) oraz podpisane słowo (w pliku `word.txt.sig`).
- (d) Zweryfikuj podpis (plik `word.txt.sig`) za pomocą pobranego klucza publicznego (`public_key.asc`). W przypadku, gdy weryfikacja się powiedzie, powinieneś zobaczyć adres e-mail osoby, która podpisała plik.
- (e) Wyślij request do endpointa <http://127.0.0.1:5012/verify> używając metody HTTP POST i prześlij do serwera id sesji (jako `session_id`) oraz adres e-mail podpisującego (jako tekst, `signer_email`).

UWAGI:

- Aby zapisać odpowiedź do pliku z rozszerzeniem *.zip, użyj opcji -o

```
[user@parrot]~  
└─$ curl -s -o data.zip -X GET http://127.0.0.1:5012/detach-sign  
[user@parrot]~  
└─$ unzip data.zip  
Archive: data.zip  
  inflating: word.txt  
  inflating: word.txt.sig  
  inflating: public_key.asc  
  inflating: session_id.txt  
[user@parrot]~  
└─$ cat session_id.txt  
80423e04ad59ba19c7d95ce02706d7c7 [user@parrot]~  
└─$ gpg --import public_key.asc  
gpg: key EEA3632AA84C26F6: public key "Grimes <GRIMES@secure.com>" imported  
gpg: Total number processed: 1  
gpg: imported: 1  
[user@parrot]~  
└─$ gpg --verify word.txt.sig  
gpg: assuming signed data in 'word.txt'  
gpg: Signature made Sat Nov 22 13:52:47 2025 UTC  
gpg: using RSA key 2EFBD23824A190958548D8A8EEA3632AA84C26F6  
gpg: Good signature from "Grimes <GRIMES@secure.com>" [unknown]  
gpg: WARNING: This key is not certified with a trusted signature!  
gpg: There is no indication that the signature belongs to the owner.  
Primary key fingerprint: 2EFB D238 24A1 9095 8548 D8A8 EEA3 632A A84C 26F6  
[user@parrot]~  
└─$ curl -X POST http://localhost:5012/verify -F "session_id=80423e04ad59ba19c7d95ce02706d7c7" -F "signer_email=GRIMES@secure.com"  
{  
  "status": "success",  
  "message": "🎉 Gratulacje! Poprawnie zweryfikowałeś oddzielny podpis i zidentyfikowałeś podpisującego!",  
  "signer_email": "GRIMES@secure.com",  
  "word": "HARDY",  
  "note": "Udowodniłeś że potrafisz zweryfikować detached signature GPG i wydobyć informacje o podpisującym"  
}[user@parrot]~
```

5.13 Pod adresem <http://127.0.0.1:5013> działa prosty serwer HTTP udostępniający 2 endpointy: <http://127.0.0.1:5013/detach-sign> oraz <http://127.0.0.1:5012/verify>.

(a) Uruchom serwer za pomocą poniższego polecenia:

```
docker run -p 5013:5013 --name ex13 docker.io/mazurkatarzyna/gpg-ex13:latest
podman run -p 5013:5013 --name ex13 docker.io/mazurkatarzyna/gpg-ex13:latest
```

Jeśli Docker Hub nie odpowiada, użyj obrazu zapasowego:

```
docker run -p 5013:5013 --name ex13 ghcr.io/mazurkatarzynaumcs/gpg-ex13:latest
podman run -p 5013:5013 --name ex13 ghcr.io/mazurkatarzynaumcs/gpg-ex13:latest
```

- (b) Wyślij request do endpointa <http://127.0.0.1:5013/clearsign> używając metody HTTP GET i zapisz odpowiedź do pliku z rozszerzeniem *.zip.
- (c) Rozpakuj pobrane archiwum. W odpowiedzi od serwera otrzymałeś id sesji (jako `session_id`), klucz publiczny GPG (`public_key.asc`) oraz podpisane słowo (w pliku `word.txt.asc`).
- (d) Zweryfikuj podpis (plik `word.txt.asc`) za pomocą pobranego klucza publicznego (`public_key.asc`). W przypadku, gdy weryfikacja się powiedzie, powinieneś zobaczyć adres e-mail osoby, która podpisała plik.
- (e) Wyślij request do endpointa <http://127.0.0.1:5013/verify> używając metody HTTP POST i prześlij do serwera id sesji (jako `session_id`) oraz adres e-mail podpisującego (jako tekst, `signer_email`).

UWAGI:

- Aby zapisać odpowiedź do pliku z rozszerzeniem *.zip, użyj opcji `-o`

```
[user@parrot]-[~]
└─ $curl -s -o data.zip -X GET http://127.0.0.1:5013/clearsign
[user@parrot]-[~]
└─ $unzip data.zip
Archive: data.zip
  inflating: word.txt.asc
replace public_key.asc? [y]es, [n]o, [A]ll, [N]one, [r]ename: A
  inflating: public_key.asc
  inflating: session_id.txt
[user@parrot]-[~]
└─ $cat session_id.txt
0147f581aad9d930ccffe03543483fe1 [user@parrot]-[~]
└─ $gpg --import public_key.asc
gpg: key 691D794B4262B855: public key "Washington <WASHINGTON@example.com>" imported
gpg: Total number processed: 1
gpg:          imported: 1
[user@parrot]-[~]
└─ $gpg --verify word.txt.asc
gpg: Signature made Sat Nov 22 13:56:01 2025 UTC
gpg:          using RSA key B7A071B39CD3AFAEF9D07085691D794B4262B855
gpg: Good signature from "Washington <WASHINGTON@example.com>" [unknown]
gpg: WARNING: This key is not certified with a trusted signature!
gpg:          There is no indication that the signature belongs to the owner.
Primary key fingerprint: B7A0 71B3 9CD3 AFAE F9D0 7085 691D 794B 4262 B855
gpg: WARNING: not a detached signature; file 'word.txt' was NOT verified!
[user@parrot]-[~]
└─ $curl -X POST http://localhost:5013/verify -F "session_id=0147f581aad9d930ccffe03543483fe1" -F "s
igner_email=WASHINGTON@example.com"
{
  "status": "success",
  "message": "🎉 Gratulacje! Poprawnie zweryfikowałeś clear-signed plik i zidentyfikowałeś podpisujące
go!",
  "signer_email": "WASHINGTON@example.com",
  "word": "PATTERSON",
  "note": "Udowodniłeś że potrafisz zweryfikować clearsign GPG i wydobyć informacje o podpisującym"
} [user@parrot]-[~]
```

5.14 Pod adresem <http://127.0.0.1:5014> działa prosty serwer HTTP udostępniający 2 endpointy: <http://127.0.0.1:5014/getkey> oraz <http://127.0.0.1:5014/submit>.

(a) Uruchom serwer za pomocą poniższego polecenia:

```
docker run -p 5014:5014 --name ex14 docker.io/mazurkatarzyna/gpg-ex14:latest  
podman run -p 5014:5014 --name ex14 docker.io/mazurkatarzyna/gpg-ex14:latest
```

Bezpieczeństwo Systemów Komputerowych - GNU Privacy Guard (GPG) | Katarzyna Mazur

Jeśli Docker Hub nie odpowiada, użyj obrazu zapasowego:

```
docker run -p 5014:5014 --name ex14 ghcr.io/mazurkatarzynaumcs/gpg-ex14:latest  
podman run -p 5014:5014 --name ex14 ghcr.io/mazurkatarzynaumcs/gpg-ex14:latest
```

- (b) Wyślij request do endpointa <http://127.0.0.1:5014/getkey> używając metody HTTP GET i zapisz odpowiedź do pliku z rozszerzeniem *.zip.
- (c) Rozpakuj pobrane archiwum. W odpowiedzi od serwera otrzymałeś id sesji (jako `session_id`) oraz klucz publiczny serwera (`server_key.asc`).
- (d) Podpisz klucz publiczny serwera swoim kluczem prywatnym. Wyeksportuj podpisany klucz serwera do pliku `signed_key.asc`.
- (e) Wyślij request do endpointa <http://127.0.0.1:5014/submit> używając metody HTTP POST i prześlij do serwera id sesji (jako `session_id`), swój klucz publiczny (jako plik, `your_pubkey`) oraz podpisany swoim kluczem prywatnym klucz publiczny serwera (jako plik, `signed_key`).

```
[user@parrot]~  
└─$ curl -s -o data.zip -X GET http://127.0.0.1:5014/getkey  
[user@parrot]~  
└─$ unzip data.zip  
Archive: data.zip  
replace server_key.asc? [y]es, [n]o, [A]ll, [N]one, [r]ename: A  
  inflating: server_key.asc  
  inflating: session_id.txt  
[user@parrot]~  
└─$ cat session_id.txt  
bcc9b413e466e9f96d0f6ff1d79aaa04 [user@parrot]~  
└─$ gpg --full-generate-key  
gpg (GnuPG) 2.2.40; Copyright (C) 2022 g10 Code GmbH  
This is free software: you are free to change and redistribute it.  
There is NO WARRANTY, to the extent permitted by law.  
  
Please select what kind of key you want:  
  (1) RSA and RSA (default)  
  (2) DSA and Elgamal  
  (3) DSA (sign only)  
  (4) RSA (sign only)  
  (14) Existing key from card  
Your selection? 1  
RSA keys may be between 1024 and 4096 bits long.  
What keysize do you want? (3072) 1024  
Requested keysize is 1024 bits  
Please specify how long the key should be valid.  
    0 = key does not expire  
    <n> = key expires in n days  
    <n>w = key expires in n weeks  
    <n>m = key expires in n months  
    <n>y = key expires in n years  
Key is valid for? (0) 1  
Key expires at Sun Nov 23 14:17:55 2025 UTC  
Is this correct? (y/N) y
```

```

pub  rsa1024 2025-11-22 [SC] [expires: 2025-11-23]
    6598465D95EFED604975140C03B108C381770DAA
uid                               student14 <student14@mail.com>
sub  rsa1024 2025-11-22 [E] [expires: 2025-11-23]

[user@parrot]~$
→ $gpg --armor --export 6598465D95EFED604975140C03B108C381770DAA > pub.key
[user@parrot]~$
→ $gpg --import server_key.asc
gpg: key 5D8236563918E4EC: public key "Lang <LANG@mail.com>" imported
gpg: Total number processed: 1
gpg:             imported: 1
[user@parrot]~$
→ $gpg --list-keys

```

```

pub  rsa1024 2025-11-22 [SC] [expires: 2025-11-23]
    6598465D95EFED604975140C03B108C381770DAA
uid          [ultimate] student14 <student14@mail.com>
sub  rsa1024 2025-11-22 [E] [expires: 2025-11-23]

pub  rsa2048 2025-11-22 [SCEA]
    8264FCBF0EAC75A393E9D9B25D8236563918E4EC
uid          [ unknown] Lang <LANG@mail.com>

[user@parrot]~$
→ $gpg --local-user 6598465D95EFED604975140C03B108C381770DAA --sign-key 8264FCBF0EAC75A393E9D9B25D8236563918E4EC

pub  rsa2048/5D8236563918E4EC
    created: 2025-11-22  expires: never      usage: SCEA
    trust: unknown      validity: unknown
[ unknown] (1). Lang <LANG@mail.com>

pub  rsa2048/5D8236563918E4EC
    created: 2025-11-22  expires: never      usage: SCEA
    trust: unknown      validity: unknown
Primary key fingerprint: 8264 FCBF 0EAC 75A3 93E9  D9B2 5D82 3656 3918 E4EC

    Lang <LANG@mail.com>

Are you sure that you want to sign this key with your
key "student14 <student14@mail.com>" (03B108C381770DAA)

Really sign? (y/N) y

```

```
[user@parrot]~$ gpg --export --armor 8264FCBF0EAC75A393E9D9B25D8236563918E4EC > signed_key.asc
[user@parrot]~$ curl -X POST http://localhost:5014/submit -F "session_id=bcc9b413e466e9f96d0f6ff1d79aaa04" -F "your_pubkey=@pub.key" -F "signed_key=@signed_key.asc"
{
  "status": "success",
  "message": "🎉 Gratulacje! Poprawnie podpisałeś klucz serwera!",
  "server_email": "LANG@mail.com",
  "signer_info": "sig          03B108C381770DAA 2025-11-22 student14 <student14@mail.com>",
  "note": "Właśnie uczestniczyłeś w Web of Trust - sieci zaufania GPG!",
  "signatures_found": 2,
  "signer_email": "student14@mail.com",
  "signer_key_id": "03B108C381770DAA"
}
[user@parrot]~$
```

5.15 Wyświetl wszystkie klucze publiczne i prywatne z Twojego keyring'a (keyring inaczej nazywany jest repozytorium kluczy programu gpg).

Wszystkie klucze (publiczne + prywatne)

```
gpg --list-keys
```

```
pub   rsa1024 2025-11-22 [SC] [expires: 2025-11-23]
       6598465D95EFED604975140C03B108C381770DAA
uid           [ultimate] student14 <student14@mail.com>
sub   rsa1024 2025-11-22 [E] [expires: 2025-11-23]
```

Tylko klucze publiczne

```
gpg --list-public-keys
```

Tylko klucze prywatne

```
gpg --list-secret-keys
```

Jeśli chcesz zobaczyć klucze z fingerprintami:

```
gpg --list-keys --fingerprint
```

```
pub  rsa1024 2025-11-22 [SC] [expires: 2025-11-23]
     6598 465D 95EF ED60 4975 140C 03B1 08C3 8177 0DAA
uid          [ultimate] student14 <student14@mail.com>
sub  rsa1024 2025-11-22 [E] [expires: 2025-11-23]
```

Jeśli chcesz format „długi” (pokazuje fingerprint):

```
gpg --list-keys --keyid-format long
gpg --list-secret-keys --keyid-format long
```

```
pub  rsa1024/03B108C381770DAA 2025-11-22 [SC] [expires: 2025-11-23]
     6598465D95EFED604975140C03B108C381770DAA
uid          [ultimate] student14 <student14@mail.com>
sub  rsa1024/DD24BFFABD730BCD 2025-11-22 [E] [expires: 2025-11-23]
```

5.16 Utwórz parę kluczy GPG przy użyciu algorytmu RSA. Następnie usuń parę kluczy (publiczny i prywatny) z Twojego keyringa.

Komendy

1. Usuwanie klucza prywatnego

```
gpg --delete-secret-keys [id klucza / mail]
```

2. Usuwanie klucza publicznego

```
gpg --delete-keys [id klucza / mail]
```

lub wszystko za jednym razem

```
gpg --delete-secret-and-public-key [id klucza / mail]
```

```
[user@parrot]-[~]  
└─ $gpg --list-keys
```

```
pub   rsa2048 2025-11-22 [SCEA]  
      8264FCBF0EAC75A393E9D9B25D8236563918E4EC  
uid           [ full ] Lang <LANG@mail.com>
```

```
[user@parrot]-[~]  
└─ $gpg --full-generate-key
```

```
pub   rsa1024 2025-11-22 [SC] [expires: 2025-11-23]  
      7E48DCE7F93B7F75A9EF015492632648F9A10E9B  
uid           student16 <student16@mail.com>  
sub   rsa1024 2025-11-22 [E] [expires: 2025-11-23]
```

```
[user@parrot]~  
$gpg --list-keys
```

```
pub  rsa2048 2025-11-22 [SCEA]  
      8264FCBF0EAC75A393E9D9B25D8236563918E4EC  
uid          [ full ] Lang <LANG@mail.com>  
  
pub  rsa1024 2025-11-22 [SC] [expires: 2025-11-23]  
      7E48DCE7F93B7F75A9EF015492632648F9A10E9B  
uid          [ultimate] student16 <student16@mail.com>  
sub  rsa1024 2025-11-22 [E] [expires: 2025-11-23]
```

```
[user@parrot]~  
$gpg --list-public-keys
```

```
pub  rsa2048 2025-11-22 [SCEA]  
      8264FCBF0EAC75A393E9D9B25D8236563918E4EC  
uid          [ full ] Lang <LANG@mail.com>  
  
pub  rsa1024 2025-11-22 [SC] [expires: 2025-11-23]  
      7E48DCE7F93B7F75A9EF015492632648F9A10E9B  
uid          [ultimate] student16 <student16@mail.com>  
sub  rsa1024 2025-11-22 [E] [expires: 2025-11-23]
```

```
[user@parrot]~  
$gpg --list-secret-keys
```

```
sec  rsa1024 2025-11-22 [SC] [expires: 2025-11-23]  
      6598465D95EFED604975140C03B108C381770DAA  
uid          [ultimate] student14 <student14@mail.com>  
ssb  rsa1024 2025-11-22 [E] [expires: 2025-11-23]  
  
sec  rsa1024 2025-11-22 [SC] [expires: 2025-11-23]  
      7E48DCE7F93B7F75A9EF015492632648F9A10E9B  
uid          [ultimate] student16 <student16@mail.com>  
ssb  rsa1024 2025-11-22 [E] [expires: 2025-11-23]
```

```
[x]-[user@parrot]-[~]  
$gpg --delete-secret-keys 7E48DCE7F93B7F75A9EF015492632648F9A10E9B  
gpg (GnuPG) 2.2.40; Copyright (C) 2022 g10 Code GmbH  
This is free software: you are free to change and redistribute it.  
There is NO WARRANTY, to the extent permitted by law.
```

```
sec  rsa1024/92632648F9A10E9B 2025-11-22 student16 <student16@mail.com>
```

```
Delete this key from the keyring? (y/N) y
```

```
This is a secret key! - really delete? (y/N) y
```

```
[user@parrot]-[~]  
$gpg --list-secret-keys
```

```
sec  rsa1024 2025-11-22 [SC] [expires: 2025-11-23]  
      6598465D95EFED604975140C03B108C381770DAA  
uid          [ultimate] student14 <student14@mail.com>  
ssb  rsa1024 2025-11-22 [E] [expires: 2025-11-23]
```

```
[user@parrot]-[~]  
$gpg --delete-key 7E48DCE7F93B7F75A9EF015492632648F9A10E9B  
gpg (GnuPG) 2.2.40; Copyright (C) 2022 g10 Code GmbH  
This is free software: you are free to change and redistribute it.  
There is NO WARRANTY, to the extent permitted by law.
```

```
pub  rsa1024/92632648F9A10E9B 2025-11-22 student16 <student16@mail.com>
```

```
Delete this key from the keyring? (y/N) y
```

```
[user@parrot]-[~]  
$gpg --list-keys
```

```
pub  rsa2048 2025-11-22 [SCEA]  
      8264FCBF0EAC75A393E9D9B25D8236563918E4EC  
uid          [ full ] Lang <LANG@mail.com>
```

```
[user@parrot]-[~]  
$gpg --list-public-keys
```

```
pub  rsa2048 2025-11-22 [SCEA]  
      8264FCBF0EAC75A393E9D9B25D8236563918E4EC  
uid          [ full ] Lang <LANG@mail.com>
```

```
[user@parrot]-[~]
```


5.17 Użytkownicy systemów linuksowych mają możliwość pobrania oprogramowania z repozytoriów przygotowanych dla danej dystrybucji systemu. Niekiedy jednak dodatkowe oprogramowanie można pobrać jedynie ze strony WWW. W takim przypadku, jaka jest pewność, że plik znajdujący się na stronie, został na niej w rzeczywistości umieszczony przez dewelopera aplikacji, a nie hakera? Niektórzy programiści podpisują swoje oprogramowanie przy pomocy rozwiązań PGP (takich jak np. GPG). Dzięki temu, jako użytkownicy, mamy możliwość weryfikacji integralności oprogramowania. Proces weryfikacji jest prosty, należy:

- (a) Pobrać klucz publiczny autora oprogramowania
- (b) Sprawdzić fingerprint klucza
- (c) Zaimportować klucz do własnego keyringa
- (d) Pobrać sygnaturę dla ściąganego oprogramowania
- (e) Użyć klucza publicznego do zweryfikowania podpisu

Pobierz oprogramowanie [VeraCrypt](#) i zweryfikuj jego integralność.

5.18 Pobierz najnowszą [wersję serwera Apache](#) i zweryfikuj integralność pobranego pliku. Potrzebny serwer kluczy to [pgpkeys.mit.edu](#). Podpisz [klucz publiczny Apache](#) dwoma kluczami z własnego keyringu.

Bardzo istotna jest pewność, że klucz publiczny, który właśnie w ten czy inny sposób pozyskałismy, należy naprawdę do osoby, do której wydaje się należeć. Zapewnieniu tego służą certyfikaty kluczy. Certyfikowanie (podpisywanie) czyjegoś klucza to nic innego, jak sygnowanie go swoim własnym - ma to na celu potwierdzenie jego autentyczności. Jeśli użytkownik A otrzyma klucz publiczny użytkownika B bezpośrednio od niego, to zapewne jest to naprawdę klucz użytkownika B. Ale jeśli otrzymuje go od użytkownika C, któremu niekoniecznie ufa, i podejrzewa, że w rzeczywistości klucz ten może być sfalszowany przez C? Cóż, jeśli klucz ten jest certyfikowany przez pewnego innego użytkownika D (któremu A ufa, i którego klucz publiczny już ma), i sygnatura się zgadza, to A zyskuje pewność, że klucz przekazany przez C jest zgodny z oryginałem. Oczywiście A zakłada w tym momencie, że D w chwili certyfikowania tego klucza miał 100% pewności, że klucz ten należy do B - albo dlatego, że otrzymał go od B bezpośrednio, albo dlatego, że klucz był certyfikowany przez kolejną osobę, do której D miał zaufanie. Należy z tego wysnuć jeden wniosek: NIGDY nie certyfikuj cudzego klucza, jeśli nie masz pewności, że nie jest on fałszywy. W przeciwnym razie osoby, które następnie ten klucz od Ciebie otrzymają, będą sądzić, że taką pewność miałeś i używać (być może fałszywego) klucza z powodu zaufania do Ciebie.